

GRAPHIC ERA (DEEMED TO BE UNIVERSITY)

Department of Computer Science and Engineering

ASSIGNMENT – UNIT II

Data Structures Using C

Queues and Linked Lists

Name:	University Roll No.:
Section:	Date of Submission:

Topics Covered: Queues, Circular Queues, Priority Queues, Singly Linked Lists, Circular Linked Lists, Linked List Representation using Arrays, Doubly Linked Lists, Two-Way Header Linked Lists, Linked Stack and Queue, and Polynomial Representation and Addition.

Instructions: Attempt all questions. Draw neat diagrams wherever required. For programming questions, write proper algorithms/C programs and mention the time complexity wherever applicable.

Questions

1. Define a **queue**. Explain the FIFO principle with a suitable example.
2. Write an algorithm to perform **ENQUEUE** and **DEQUEUE** operations on a queue implemented using an array. State the **overflow** and **underflow** conditions.
3. Consider a queue of size 6. Initially,

$$FRONT = REAR = -1$$

Perform the following operations:

*ENQUEUE(10), ENQUEUE(20), ENQUEUE(30),
DEQUEUE(), ENQUEUE(40), ENQUEUE(50), DEQUEUE()*

Show the values of **FRONT**, **REAR**, and the contents of the queue after each operation.

4. What is the major limitation of a **linear queue implemented using an array**? Explain how a **circular queue** solves this problem.
5. Write a C program to implement a **circular queue** using an array. Your program must support the following operations:
 - (a) Insertion
 - (b) Deletion

(c) Display

6. A circular queue has a maximum size of 5. Starting with an empty queue, perform the following operations:

*ENQUEUE(11), ENQUEUE(22), ENQUEUE(33),
DEQUEUE(), ENQUEUE(44), ENQUEUE(55), ENQUEUE(66)*

Draw the circular queue after each operation and clearly indicate **FRONT** and **REAR**.

7. Derive and explain the **full** and **empty** conditions for a circular queue implemented using an array.
8. Define a **priority queue**. How is it different from an ordinary queue? Explain how elements having the **same priority** should be processed.
9. Compare **sorted** and **unsorted** implementations of a priority queue in terms of insertion and deletion complexity.
10. Design a C structure for implementing a **priority queue using a linked list**. Write the algorithm for inserting an element according to its priority.
11. Define a **singly linked list**. Draw the linked representation of the following sequence:

10, 25, 40, 55

Clearly show **START**, **DATA**, **NEXT**, and **NULL**.

12. Consider the following memory representation of a linked list:

Address	DATA	NEXT
100	25	700
300	40	NULL
500	10	100
700	35	300

If

START = 500

determine the logical order of elements in the linked list. Explain why the **physical memory order** and **logical order** of the nodes can be different.

13. Write a C function to **traverse a singly linked list** and count the total number of nodes. Analyse the time complexity of the algorithm.
14. Write algorithms for inserting a node in a singly linked list:
- (a) At the beginning
- (b) At the end

- (c) After a specified node

15. Consider the following singly linked list:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$$

Perform the following operations sequentially and draw the list after each operation:

- (a) Insert 5 at the beginning.
 - (b) Insert 25 after 20.
 - (c) Delete 30.
 - (d) Insert 50 at the end.
16. Write a C function to **search for a given element** in a singly linked list. State its **best-case** and **worst-case** time complexities.
17. Write algorithms to perform the following operations on a singly linked list:
- (a) Delete the first node.
 - (b) Delete the last node.
 - (c) Delete a specified node.

Also explain the role of the **free()** function during deletion.

18. What is a **circular linked list**? Compare its termination condition with that of a linear singly linked list.

Why can the following condition cause an infinite loop when used incorrectly for a circular linked list?

$$\text{while}(\text{ptr} \neq \text{NULL})$$

19. Write a C function to traverse and display a **circular singly linked list** using a **do-while** loop.
20. Consider the circular linked list:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow \text{START}$$

Perform the following operations and draw the circular linked list after each operation:

- (a) Insert 5 at the beginning.
- (b) Insert 40 at the end.
- (c) Delete the first node.
- (d) Delete the last node.

21. Explain how a **linked list can be represented using an array**. What information is stored in the **NEXT** field in an array-based linked-list representation?
22. Consider the following array-based linked list:

Index	DATA	NEXT
0	50	-1
1	30	0
2	10	4
3	40	1
4	20	3

If

$$START = 2$$

determine:

- (a) The logical linked list.
 - (b) The sequence of array indices followed during traversal.
23. What is a **doubly linked list**? Draw the representation of

$$10, 20, 30, 40$$

showing the **PREV**, **DATA**, and **NEXT** fields of each node.

24. Write the C structure for a **doubly linked-list node**. Write algorithms for:
- (a) Forward traversal
 - (b) Backward traversal
25. Consider the doubly linked list:

$$NULL \leftarrow 10 \leftrightarrow 20 \leftrightarrow 30 \rightarrow NULL$$

Insert a new node containing **25** between 20 and 30.

Draw the list before and after insertion and identify **all pointer fields** that must be modified.

26. Explain insertion and deletion at the **beginning** and **end** of a doubly linked list.
Why can the **PREV** pointer make certain deletion operations easier than in a singly linked list?
27. What is a **two-way header linked list**? Explain the purpose of a header node.
How can a **circular two-way header list** simplify the handling of:
- (a) An empty list
 - (b) The first node

(c) The last node

28. Explain how a **stack** and a **queue** can be implemented using linked lists.

Clearly state which end or pointer should be used for the following operations:

(a) PUSH

(b) POP

(c) ENQUEUE

(d) DEQUEUE

Also discuss the time complexity of these operations.

29. Represent the polynomial

$$P(x) = 6x^3 + 9x^2 + 7x + 1$$

using a linked list.

Clearly show the **coefficient**, **exponent**, and **NEXT** fields of every node.

Also write the corresponding C structure for representing a polynomial term.

30. Consider the following two polynomials:

$$P(x) = 5x^4 + 3x^2 + 2x + 7$$

and

$$Q(x) = 4x^3 + 6x^2 + 5$$

Perform the following:

(a) Represent both polynomials using linked lists.

(b) Add the two polynomials using the **two-pointer technique**.

(c) Draw the resulting polynomial linked list.

(d) Write an algorithm or C function to perform polynomial addition.

(e) Analyse the time complexity of polynomial addition in terms of the number of terms in the two polynomials.

— End of Assignment —

Students are expected to draw all linked-list and queue representations clearly and mention pointer/index changes wherever applicable.